# FINNEWS AI

## Financial News Simplifier

*"Understand financial news without the jargon."*

## Project Report and Documentation

AI-powered financial news simplifier built with FastAPI, NewsAPI and Groq LLM (LLaMA 3.3 70B)

Repository: github.com/prajwalarote05-tech/FinNews-AI

Developer: Prajwal Arote

# Table of Contents

1. Abstract
2. Introduction and Problem Statement
3. Objectives
4. Proposed Solution
5. Key Features
6. Technology Stack
7. System Architecture
8. Project Structure and Module Description
9. API Documentation
10. AI Design and Guardrails
11. Installation and Setup
12. Running the Application
13. Application Workflow
14. Testing
15. Limitations
16. Future Enhancements
17. Conclusion
18. Disclaimer

# 1. Abstract

FinNews AI is a full-stack capstone web application that fetches real-time financial and business news and converts complex market reports into clear, beginner-friendly explanations. The backend, built on FastAPI, retrieves headlines from NewsAPI, cleans and deduplicates them, and sends the text to Groq's LLM inference service running LLaMA 3.3 70B. The model returns a structured JSON response containing a simple summary, key takeaways, a "Why It Matters" explanation and a glossary of financial terms. A lightweight HTML, CSS and vanilla JavaScript dashboard displays the results as readable cards, with no build step or heavy frontend framework required.

# 2. Introduction and Problem Statement

Financial news outlets often rely on dense terminology such as *quantitative tightening, basis points, yield inversion, EBITDA compression* and *liquidity crunches*. For college students, beginner investors, non-finance professionals and busy readers, this creates several problems:

- Articles are intimidating and time-consuming to read.
- Essential takeaways get obscured by market jargon.
- Misinterpretation can lead to anxiety or poor financial decisions.
- Existing AI tools often over-engineer responses or provide risky, unwarranted financial advice.

# 3. Objectives

- Fetch current financial news across stock market, economy, business and banking sectors.
- Clean and deduplicate articles so users see reliable, unique stories.
- Simplify each article into plain English without losing exact figures, dates, percentages or company names.
- Prevent investment advice, stock predictions and hallucinated content through strict prompt guardrails.
- Deliver a responsive, accessible dashboard that works on desktop, tablet and mobile.
- Provide automated tests and a one-click startup experience for easy evaluation.

# 4. Proposed Solution

FinNews AI offers a streamlined, single-click solution. It pulls real-time business and market news from NewsAPI, applies an AI prompt with strict guardrails, and breaks each story into four easy components:

| Component | Description |
|---|---|
| Simple Summary | 2-3 sentences explaining the story in plain English. |
| Key Takeaways | Three bullet points highlighting the main facts. |
| Why It Matters | A relatable explanation of real-world impact on everyday consumers and investors. |
| Financial Terms Explained | A pocket glossary defining the tricky words used in the article. |

## 5. Key Features

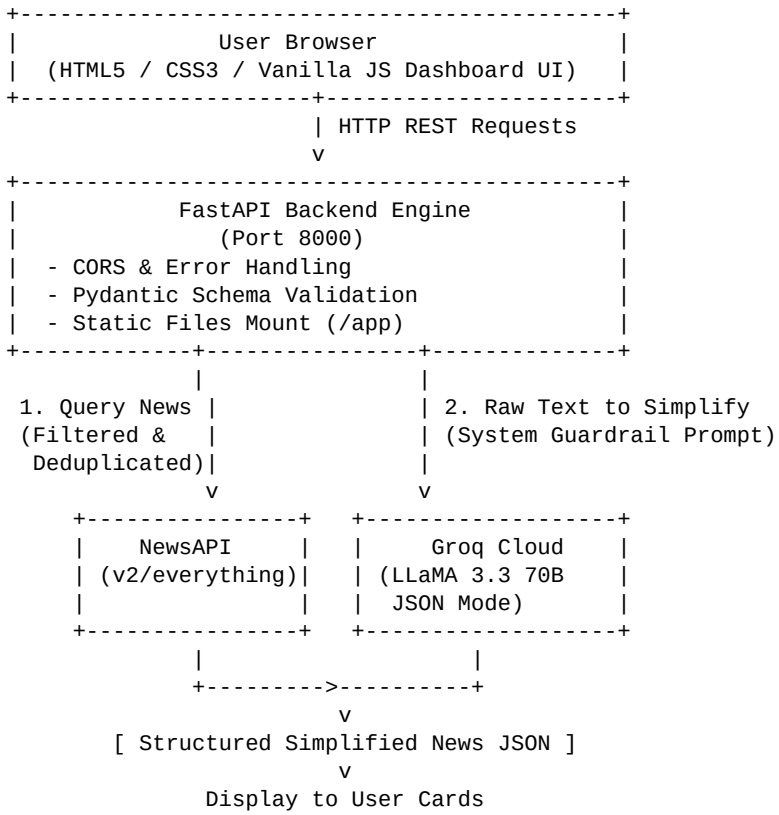
- **One-Click Simplification:** the "Fetch & Simplify News" button processes the top stories automatically.
- **Topic Filtering:** filter by All, Stock Market, Economy, Business or Finance.
- **Real-Time Deduplication:** removes duplicate wire stories, broken or withdrawn articles, and sanitizes truncated feeds.
- **Configurable Groq LLM:** the model (default `llama-3.3-70b-versatile`) is changed through an environment variable, with no code changes.
- **Strict JSON Output:** Groq's JSON mode ensures consistent parsing and UI rendering.
- **Responsive Dashboard:** a dark-mode financial/AI interface for all screen sizes.
- **Automated Windows Script (run.bat):** creates the virtual environment, installs dependencies, initializes .env and launches Uvicorn.
- **Interactive Swagger Docs:** full OpenAPI exploration available at /docs.

## 6. Technology Stack

| Layer | Technology | Purpose |
|---|---|---|
| Language | Python 3.12+ (tested on 3.13) | Core backend language |
| Web framework | FastAPI | High-performance asynchronous API |
| Server | Uvicorn | ASGI web server |
| HTTP client | httpx | Async requests to NewsAPI with timeout management |
| AI SDK | groq | Official Python SDK for Groq LLM inference |
| Validation | pydantic v2 | Request and response data validation |
| Configuration | python-dotenv | Environment variable management |
| LLM | LLaMA 3.3 70B Versatile via Groq Cloud | Summarization and simplification |
| News source | NewsAPI | Real-time global financial and business news |
| Frontend | HTML5, CSS3, Vanilla JavaScript (ES6+) | Dashboard UI with Grid, Flexbox and custom properties; no build step |
| Testing | pytest, pytest-asyncio, FastAPI TestClient | Automated tests with mocked external services |

## 7. System Architecture

The system follows a three-tier design. The browser-based dashboard sends REST requests to the FastAPI backend, which handles CORS, schema validation and static file hosting. The backend queries NewsAPI for articles, then passes the cleaned text to Groq Cloud together with a guardrail system prompt. The resulting structured JSON is returned to the browser and rendered as cards.




# 8. Project Structure and Module Description

```
finnews-ai/
|-- backend/
|   |-- main.py            # FastAPI app, endpoints, CORS, static hosting
|   |-- config.py          # Environment variable configuration
|   |-- news_service.py    # NewsAPI fetcher, query mapper, deduplication
|   |-- ai_service.py      # Groq client, prompt, JSON parser/validator
|   |-- schemas.py         # Pydantic request/response models
|   |-- requirements.txt   # Python dependencies
|   `-- .env.example       # Environment template
|-- frontend/
|   |-- index.html         # User interface structure
|   |-- style.css          # Dark financial-tech styling
|   `-- script.js          # State, API integration, card renderer
|-- tests/
|   |-- test_health.py     # Health and root endpoint tests
|   `-- test_api.py        # News and simplify tests with mocks
|-- .gitignore
|-- README.md
|-- pytest.ini
`-- run.bat                # One-click startup for Windows
```

| Module | Responsibility |
|---|---|
| main.py | Creates the FastAPI application, defines all endpoints, configures CORS and error handling, and mounts the frontend at /app. |
| config.py | Loads and validates environment variables (NEWS_API_KEY, GROQ_API_KEY, GROQ_MODEL). |
| news_service.py | Calls NewsAPI, maps topics to search queries, and removes duplicate, broken or truncated articles. |

| Module | Responsibility |
|---|---|
| ai_service.py | Initializes the Groq client, holds the guardrail prompt, requests JSON-mode output, then parses and validates it. |
| schemas.py | Pydantic models that define and validate request and response structures. |
| frontend/ | Static dashboard: page structure (HTML), styling and responsiveness (CSS), and API calls plus dynamic card rendering (JS). |

# 9. API Documentation

| Method | Endpoint | Description | Parameters |
|---|---|---|---|
| GET | / | API status and links | None |
| GET | /health | Service health status | None |
| GET | /api/news | Retrieve raw deduplicated news | ?topic=all&page_size=5 |
| POST | /api/simplify | Simplify a single article | JSON SimplifyRequest body |
| POST | /api/news/simplify-all | Batch fetch and simplify news | ?topic=all&limit=3 |

### Sample request: POST /api/simplify

```
{
  "title": "Fed Holds Benchmark Rate at 5.25%-5.50% Amid Sticky Inflation",
  "description": "Federal Reserve policymakers voted unanimously to keep rates steady.",
  "content": "The Federal Open Market Committee left its benchmark policy rate unchanged...",
  "source": "Bloomberg",
  "publishedAt": "2026-09-25T14:30:00Z",
  "url": "https://bloomberg.com/news/fed-rate-decision"
}
```

### Sample response

```
{
  "title": "Fed Holds Benchmark Rate at 5.25%-5.50% Amid Sticky Inflation",
  "simple_summary": "The Federal Reserve decided not to raise or lower interest
     rates today. Officials want more proof that price increases are cooling...",
  "key_points": [
    "U.S. central bank keeps interest rates unchanged at 5.25% to 5.50%.",
    "Inflation has improved but remains above the 2% target.",
    "Policymakers are waiting for clearer data before cutting rates."
  ],
  "why_it_matters": "When rates stay high, borrowing for mortgages, student
     loans and credit cards stays more expensive...",
  "terms_explained": [
    {"term": "Federal Reserve", "meaning": "The central bank of the U.S...."},
    {"term": "Benchmark Rate", "meaning": "The baseline interest rate..."}
  ],
  "source": "Bloomberg",
  "published_at": "2026-09-25T14:30:00Z",
  "url": "https://bloomberg.com/news/fed-rate-decision"
}
```

Interactive documentation for all endpoints is available through Swagger UI at `http://127.0.0.1:8000/docs`.

# 10. AI Design and Guardrails

The AI layer is designed for safety and consistency rather than open-ended generation. The system prompt instructs the model to:

- Never give investment advice, recommend buying, selling or holding any asset, or predict stock movements.
- Never hallucinate; stay faithful to the supplied article text.
- Preserve exact figures, dates, percentages and company names.
- Respond only in a fixed JSON schema (summary, key points, why it matters, terms explained).

Groq's JSON mode enforces structured output, and the backend parses and validates the response with Pydantic before it reaches the UI. The model is configurable through the `GROQ_MODEL` environment variable.

# 11. Installation and Setup

## Prerequisites

- Python 3.12 or higher (check with `python --version`).
- Git (optional).

## Steps

```
# 1. Open the project directory
cd finnews-ai

# 2. Create and activate a virtual environment
python -m venv venv
venv\Scripts\activate            (Windows)
source venv/bin/activate         (macOS / Linux)

# 3. Install dependencies
pip install -r backend/requirements.txt
```

## API keys

Two free API keys are required: a **NewsAPI key** (register at newsapi.org/register) and a **Groq API key** (create one at console.groq.com/keys). Create a `.env` file in the project root (or copy `backend/.env.example`):

```
NEWS_API_KEY=your_actual_newsapi_key_here
GROQ_API_KEY=your_actual_groq_api_key_here
GROQ_MODEL=llama-3.3-70b-versatile
```

**Security note:** never commit the .env file to GitHub. The .gitignore file already excludes it.

# 12. Running the Application

## Method 1: One-click startup on Windows (recommended)

Double-click `run.bat`. It verifies Python, creates the virtual environment if missing, installs dependencies, creates .env from the template and launches Uvicorn with auto-reload at http://127.0.0.1:8000.

## Method 2: Manual execution

```
uvicorn backend.main:app --reload --host 127.0.0.1 --port 8000
```

| Resource | URL |
|---|---|
| Web dashboard | http://127.0.0.1:8000/app/ |
| Swagger docs | http://127.0.0.1:8000/docs |
| Health endpoint | http://127.0.0.1:8000/health |

The frontend/index.html file can also be opened directly in a browser; it connects to the backend through CORS.

## 13. Application Workflow

**1.** The user opens the dashboard at /app/.

**2.** The browser pings /health in the background and shows a "Backend Connected" badge.

**3.** The user selects a topic filter, for example "Stock Market".

**4.** The user clicks "Fetch & Simplify News".

**5.** The button is disabled and the UI shows progress: "Fetching latest financial news..." then "Simplifying with AI...".

**6.** The backend queries NewsAPI, deduplicates and validates articles, then calls Groq with JSON mode and the guardrail prompt.

**7.** Cards render dynamically with badges, summary box, key takeaways, a "Why It Matters" callout and a jargon glossary.

## 14. Testing

The `tests/` directory contains an automated suite using pytest, pytest-asyncio and the FastAPI TestClient. External network calls are mocked, so tests run quickly without using live API tokens. Run the suite with `pytest -v`.

| Test | Purpose |
|---|---|
| test_root_endpoint | Verifies the root endpoint returns API status. |
| test_health_endpoint | Verifies the health check endpoint. |
| test_simplify_validation_missing_title | Confirms validation rejects requests without a title. |
| test_simplify_success | Confirms a single article is simplified correctly (mocked AI). |
| test_get_news_missing_api_key | Confirms proper error handling when the NewsAPI key is missing. |
| test_get_news_success | Confirms news retrieval and deduplication (mocked NewsAPI). |
| test_simplify_all_news_success | Confirms the batch fetch-and-simplify flow. |

Expected result: **7 passed**.

## 15. Limitations

- **NewsAPI developer tier:** the free tier limits requests to 100 queries per day and applies a 24-hour publication delay.

- **Article truncation:** free NewsAPI responses may truncate body text at 200 characters. The app combines title and description when the body is truncated.
- **Rate limits:** the free Groq tier has per-minute limits, so batch requests are clamped to 3-5 articles at a time.

## 16. Future Enhancements

- **Audio briefings (TTS):** browser SpeechSynthesis or an AI voice model to read summaries aloud.
- **Search query input:** search by company ticker (e.g., $AAPL, $TSLA) or custom keywords.
- **Bookmarks:** save favorite news cards in browser localStorage.
- **Multi-language support:** translate summaries into Spanish, Hindi, French and Mandarin.
- **Sentiment gauge:** visual indicator of positive, neutral or cautious economic sentiment.

## 17. Conclusion

FinNews AI demonstrates how a modern LLM can be combined with a real-time data source to make financial information accessible to everyone. By pairing a fast FastAPI backend, NewsAPI curation, Groq's low-latency LLaMA 3.3 70B inference and strict safety guardrails, the project turns jargon-heavy reporting into short, understandable and trustworthy summaries. Its simple setup, automated tests and clean, dependency-free frontend make it easy to run, evaluate and extend.

## 18. Disclaimer

**FinNews AI provides simplified summaries for educational and informational purposes only. It does NOT provide financial, investment, legal or tax advice.** The application does not recommend buying, selling or holding any securities, stocks, cryptocurrencies or financial instruments. Always consult a certified financial advisor before making investment decisions.